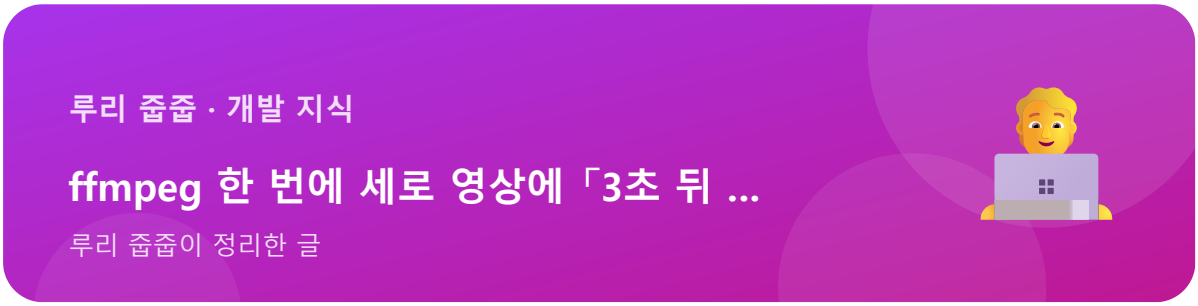


ffmpeg 한 번에 세로 영상에「3초 뒤 배너」와「끝 카드」 붙이기 (파이썬)

> 작성 2026-10-20 · 읽는 시간 약 6분 · 작성 루리 줍줍 편집

#자동화

30초 요약

ffmpeg filter_complex 한 번으로 세로 영상에 3초부터 나오는 배너와 3.5초 끝 카드를 붙이는 파이썬 코드, enable 쉼표·concat 규격·오디오 채널 실수 3가지.

이 글의 목차

1. 필터 그래프
2. 실수 1: enable= 안의 쉼표
3. 실수 2: concat이 끝 카드를 거부함
4. 실수 3: 오디오 채널 배치
5. 원본은 남겨 두기
6. 결과

파이썬으로 세로 영상(1080×1920)을 만들어 올리는 자동화를 돌리고 있습니다. 렌더러는 건드리지 않고, 렌더가 끝난 뒤에 두 가지를 더 붙이고 싶었습니다.

1. 영상 3초 지점부터 끝까지 계속 보이는 배너

2. 마지막 프레임 뒤에 붙는 3.5초짜리 정지 이미지(끝 카드). 오디오 길이를 맞추려고 무음도 같이 붙입니다.

후처리로 빼 두면 렌더러가 단순하게 유지되고 배너 내용도 나중에 바꾸기 쉽습니다. 아래는 제가 쓰는 ffmpeg 호출 한 번짜리 코드와, 가는 길에 만난 실수 세 가지입니다.

필터 그래프

입력은 네 개입니다.

- `0` 렌더가 끝난 영상(오디오 포함)
- `1` 끝 카드 PNG(3.5초 반복)
- `2` 끝 카드용 무음
- `3` 배너 PNG(모서리가 투명한 RGBA)

```
import subprocess, imageio_ffmpeg

FF = imageio_ffmpeg.get_ffmpeg_exe()   # 번들 ffmpeg, 따로 설치 안 해도 됨
DUR = 3.5

def add_banner_and_endcard(src, endcard_png, banner_png, out, w, h, fps, sr):
    sx, sy = int(w * 40 / 1080), int(h * 50 / 1920)   # 영상 크기에 맞춰 위치 계산
    sw = int(w * 1000 / 1080)
    graph = (
        rf"[3:v]scale={sw}:-1[s];"
        rf"[0:v][s]overlay={sx}:{sy}:enable=gte(t\,3)[o];"          # 3초부터 배너
        rf"[1:v]scale={w}:{h},fps={fps},format=yuv420p,setsar=1[e];"  # 끝 카드를 본편고
        rf"[o]format=yuv420p,setsar=1[m];"
        rf"[m][0:a][e][2:a]concat=n=2:v=1:a=1[v][a]"
    )
    cmd = [FF, "-y",
           "-i", src,
           "-loop", "1", "-t", str(DUR), "-i", endcard_png,
           "-f", "lavfi", "-t", str(DUR), "-i", f"anullsrc=r={sr}:cl=mono",
           "-i", banner_png,
           "-filter_complex", graph,
           "-map", "[v]", "-map", "[a]",
           "-c:v", "libx264", "-preset", "veryfast", "-crf", "20",
           "-c:a", "aac", "-b:a", "160k", "-movflags", "+faststart", out]
    subprocess.run(cmd, check=True)
```

복사

w, h, fps, 오디오 샘플레이트 sr은 원본에서 읽어 옵니다. `ffmpeg -i` 원본을 실행해 나오는 stderr를 정규식으로 훑으면 충분합니다.

실수 1: `enable=` 안의 쉼표

`enable=gte(t,3)`은 멀쩡해 보이지만 `-filter_complex` 안에서 쉼표는 필터 구분자입니다. 식이 둘로 쪼개져서 알아보기 힘든 파싱 오류가 납니다. `gte(t\,3)`처럼 쉼표를 이스케이프하면 됩니다.

파이썬에서는 그래프 문자열을 raw 문자열(`rf"..."`)로 써야 백슬래시가 ffmpeg까지 그대로 갑니다. 일반 f-string이면 오늘은 `SyntaxWarning`, 내일은 버그입니다.

실수 2: `concat` 이 끝 카드를 거부함

`concat`은 이어 붙이는 조각들의 크기, 프레임레이트, 픽셀 포맷, SAR이 모두 같아야 합니다. PNG 반복은 RGB이고 fps 정보가 없으며 SAR도 다를 수 있습니다. 양쪽 모두 `scale=…,fps=…,format=yuv420p,setsar=1`로 규격을 맞추자 "Input link parameters do not match" 오류가 모두 사라졌습니다.

실수 3: 오디오 채널 배치

제 렌더러는 24kHz 모노 오디오를 씁니다. 무음을 `cl=stereo`로 만들었더니 `concat`이 오디오가 안 맞는다고 했습니다. 무음도 원본과 같은 샘플레이트·채널(`anullsrc=r=24000:cl=mono`)로 만들거나, 원본을 먼저 읽어서 맞춰야 합니다.

원본은 남겨 두기

위 함수는 새 파일에 씁니다. 실제 파이프라인에서는 원본을 먼저 `이름.noend.mp4`로 옮기고 성공했을 때만 같은 이름으로 덮어씁니다. 중간에 프로세스가 죽어도 원본이 남아 있고, 그 파일이 있으면 "이미 처리한 영상"이라고 보고 건너뛰게 만들어 두었기 때문에 같은 작업을 두 번 돌려도 안전합니다.

결과

배너(3초부터 끝까지)와 끝 카드가 인코딩 한 번에 붙습니다. 40초짜리 영상이면 일반 데스크톱 CPU에서 10~20초 걸립니다. 「특정 시각부터 워터마크」나 「끝에 슬레이트 붙이기」 같은 작업에 그대로 쓸 수 있는 패턴입니다.

저는 정보 사이트를 AI 코딩 에이전트와 함께 운영하며 이런 자동화를 매일 돌리고 있습니다. 그 사이트는 루리의 줍줍 (https://sungprofile.cloud/)이고, 자동화·개발 기록은 개발 지식 (https://sungprofile.cloud/ailog/)에 쌓고 있습니다.

1 English summary

A single ffmpeg filter_complex call that adds a banner starting at 3 seconds and a 3.5-second end card (with matching silent audio) to a finished vertical video, driven from Python. Three mistakes are explained: the comma inside enable=gte(t,3) must be escaped, the concat inputs and audio channels must match in format, and the Python graph string must be a raw string so the backslashes reach ffmpeg.

2 관련 글

뉴스 이슈에 「내 생활 영향」과 실제 시세를 붙이는 법 — 규칙표·FRED·SVG로 지어내지 않기

영향 규칙표 10개로 이슈 60개 중 7개에 생활 영향 카드를 붙이고, FRED·업비트 공개 시세 6개(1.6초)로 전문가 발언 뒤 7일·30일 등락을 사후 확인하는 정적 사이트 파이프라인.

윈도우 11에서 wmic가 사라져 예약 작업이 조용히 멈춘 이야기 (파이썬 대체 코드)

윈도우 11에 wmic가 없어 파이썬 예약 작업이 FileNotFoundError 로 조용히 실패한 원인과, Get-CimInstance·psutil 대체 코드, 점검 실패 시 멈추게 하는 법.

방문 집계에 우리 점검 방문이 섞여 수치가 부풀었던 일 — nohit 플래그와 데이터센터 표시로 고친 과정

하루 방문 214명 중 사람일 가능성이 큰 쪽은 76명이었어요. 링크 확인 방문과 데이터센터 점검 트래픽을 가려낸 규칙, ?nohit=1 플래그, Cloudflare Workers의 ASN·접속 회사 표시, 읽음 신호까지 정리했어요.

← 이전 글

다음 글 →

윈도우 11에서 wmic가 사라져 예약 작업이 조용히 멈춘 이야기 (파이썬 대체 코드)

방문 집계에 우리 점검 방문이 섞여 수치가 부풀었던 일 — nohit 플래그와 데이터센터 표시로 고친 과정

← 기술 블로그 홈 · 사이트 제작 로그 · RSS

관련: 개발 지식 전체

이 글에 대해 묻고 싶으면 ❓ 개발 FAQ에 남겨 주세요.

📤 공유 공유하기 카카오톡 🔗 링크 복사 네이버 블로그 밴드 X Facebook

출처: 루리의 줍줍(sungprofile.cloud) · 최신 내용은 사이트에서 확인하세요.